Authorization and Permissions Lab

Summary: Reviewing and adjusting file and directory permissions is an important skill to learn because authorization determines who can access or modify sensitive information. I examined the authorization settings on multiple files and updated them to remove unnecessary access, confirming each item aligned with the intended authorization level. This lab emphasized how permission-related commands reinforce proper access controls and protect sensitive information in Linux.

Understanding Permission Strings: Linux displays file and directory permissions using a 10-character string, such as `drwxrwxr-x`. This format provides a quick way to understand what type of item you're looking at and what level of access each user type has. The first character indicates the item type, 'd' for directory or '-' for a regular file. The remaining nine characters are grouped into three sets, representing permissions for the user, the group, and others. Each set uses 'r', 'w', and 'x' to indicate read, write, and execute privileges. A hyphen in any of the last nine characters is when a permission is not granted.

Looking at the string from earlier, `drwxrwxr-x`, we know it represents a directory where the user and group have full privileges, while other users can only read and execute. Below some examples of file and subdirectory permissions:

```
researcher2@3c7a20ea557e:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Apr 24 22:00 .
drwxr-xr-x 3 researcher2 research_team 4096 Apr 24 23:14 ..
-rw--w---- 1 researcher2 research_team   46 Apr 24 22:00 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Apr 24 22:00 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Apr 24 22:00 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Apr 24 22:00 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Apr 24 22:00 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Apr 24 22:00 project_t.txt
```

Overview: I started by navigating into the working directory and listing the contents with detailed output to review the existing permission settings using the `ls -la` command. This provides each file's permissions in extensive detail, which allowed me to check how access was assigned to different user types, including any hidden files. In Linux, any item that begins with a period is considered hidden, and the only way to view these items is by using `ls -la`. This initial review helped establish which items were already configured correctly and which ones needed adjustments.

The next step was to evaluate the files and determine which ones were granted more access than intended. I used `chmod` to modify permissions on the appropriate files. For example, I removed write permissions from owner types that should not be allowed to edit certain files and restricted access on items that were meant to be user-only. Throughout this process, I relied on permission indicators in the directory listing to verify that each change aligned with the expected authorization requirements.

To complete the lab, I checked the permissions on a subdirectory that required additional restrictions and updated its settings. After applying the necessary changes, I listed the directory again to confirm that the permissions reflected the correct authorization structure. These steps show how permission management and careful review help maintain a tidy and secure Linux environment.

Reflection: Proper authorization settings are critical when managing files in a shared environment. Working with permission strings and adjusting them with `chmod` made it clearer how each access level contributes to system security. Reviewing ownership details also highlighted how permissions and ownership work together to control who can interact with specific resources.